A2 · 学生实操手册 —— SOLO 生成服务端:建库 + Socket 多线程服务端 + 打通登录链路

> **对应**:CS 实训(C/S 模式 Java 图形界面开发)· Vibe Coding 单元② **用法**:跟着一步一步做,每一步都有【操作】【预期产出】【✅ 怎么算做好】【卡住怎么办】。老师带做时你同步做,课后自己做也照这份来。 **前置**:单元①写好的 `.trae/rules` 已在工作区;本机 MySQL 8 已启动(`127.0.0.1:3306` / root / 12345678),JDK 11 就绪;`lib/` 下有 MySQL 驱动 `mysql-connector-j` jar。

今天要做什么(先读这一段)

单元①你写好了图纸(Agent PRD / `.trae/rules`)。**今天开始施工。**

施工第一步不是画界面,而是**先把 C/S 的那条命脉通道打通**:客户端发一行文本 → 服务端收到、查数据库 → 服务端返回一行文本。这条链路一通,后面查书、借书、还书全是**复制它换业务**。所以今天只做一件核心的事——**用"登录"把这条通道打通**。

今天分四步走(每步跑通再下一步)——**Step 0 先补基础**,再进图书馆三步:

步骤	你要做的事	产出
Step 0	先跑通一个最小 echo demo(补 Socket/线程基础)	服务端收到什么原样发回,亲眼看 socket 通信发生
Step 1	建库 + 种子数据	`library_cs` 库 + 四张表 + 测试数据,能 `SELECT` 出来
Step 2	服务端骨架(Socket + 多线程 + DBUtil)	服务端能启动监听 9000、每客户端一线程、收一行回显
Step 3	打通登录链路	发 `LOGIN|admin|123456` 收到 `OK|admin|系统管理员|1`
Step 4	顺手闭合注册链路(同一通道换业务)	注册新读者 `REGISTER|...` 收 `OK|id`,再用它登录收到 `OK`

> ⚠️ **动手前先看懂两块基础**:今天用到的 **Socket(网络编程)**和**多线程**你还没学过。跟做前请先看《学生版讲义》一~三节(或母本 Part 1)的速记——**IP/端口/TCP**、**`ServerSocket` / `Socket` / `accept()` /输入输出流、进程vs线程**、**`Thread` / `Runnable` / `start()`**。Step 0 的 echo demo 就是把这些落地。看不懂的地方,当场让 AI 用中文逐行解释。

> 💡 再记住一条 **C/S 心法**:先把"客户端 → 服务端 → 数据库 → 返回"**一条链路(登录)打通**,后面所有功能都是复制这条链路换业务。**别先堆界面。**

每步的"预期产出"给的是**答案版** `cs-swing-library-demo` **的真实代码片段**——AI 给你生成的应该和它八九不离十,拿它对照着看、对照着审。

Step 0 · 先跑通一个最小 echo demo(亲眼看 socket 通信)

目的:图书馆服务端 = **echo demo + 查数据库 + 每客户端一线程**。先把最小的 echo(回声:服务端收到什么原样发回)跑通,你对"socket 通信到底怎么发生"就有实感了,再看图书馆服务端就不虚。

操作

- 在 Trae SOLO 里**原样复制**下面这段输入并回车:

```
用纯 Java 11 写一个最小的 Socket echo(回声)demo,不用任何框架,分成两个可直接运行的类:
- EchoServer:用 ServerSocket 监听 9000;accept() 等一个客户端连上;从 socket 拿输入流(BufferedReader)
  和输出流(BufferedWriter),流都用 UTF-8;循环 readLine() 收一行,原样加个前缀"你说的是:"写回去,
  记得 write 换行 + flush。
- EchoClient:用 Socket 连 127.0.0.1:9000,发一行"你好,服务端",再 readLine() 读服务端回复并打印。
只要这两个类,能直接编译运行。
```

- **先运行** `EchoServer`:控制台打印"在 9000 端口等客户端连接..."后**停住不动**——这不是卡死,是 `accept()` 在**阻塞等**客户端。
- **再运行** `EchoClient`:它连上、发一句、收一句、打印。

预期产出(对照母本 Part 1.2 的最小示例)

① `EchoServer.java`(逐行注释见母本):

```
ServerSocket serverSocket = new ServerSocket(9000);    // 在 9000 端口守着等电话(开总机)
Socket socket = serverSocket.accept();                 // 阻塞等:有人连上才返回一个 Socket
BufferedReader in = new BufferedReader(                // 输入流:读对方的话(UTF-8)
    new InputStreamReader(socket.getInputStream(), StandardCharsets.UTF_8));
BufferedWriter out = new BufferedWriter(               // 输出流:写给对方(UTF-8)
    new OutputStreamWriter(socket.getOutputStream(), StandardCharsets.UTF_8));
String line;
while ((line = in.readLine()) != null) {               // 读一行(没有就停在这等)
    out.write("你说的是:" + line);                      // 原样加前缀写回去
    out.write("\n");                                   // 补换行,对方 readLine 才知道一行结束
    out.flush();                                       // flush 才真正发出去
}
```

② `EchoClient.java`:

```
Socket socket = new Socket("127.0.0.1", 9000);          // 拨通本机的 9000 房间(建
TCP 连接)
BufferedWriter out = new BufferedWriter(
    new OutputStreamWriter(socket.getOutputStream(), StandardCharsets.UTF_8));
BufferedReader in = new BufferedReader(
    new InputStreamReader(socket.getInputStream(), StandardCharsets.UTF_8));
out.write("你好,服务端"); out.write("\n"); out.flush();  // 写一行发过去
String reply = in.readLine();                            // 读服务端回的一行
System.out.println("服务端回复:" + reply);              // 期望:服务端回复:你说的是:你好,
服务端
```

✅ 怎么算做好(你该看到什么)

- 服务端控制台打印 `有客户端连上来了!` 和 `收到:你好,服务端`;客户端控制台打印 `服务端回复:你说的是:你好,服务端`。**这一刻,你亲眼看到了一次完整的"发一句、收一句"的 socket 通信。**

再体会一下"单线程只能服务一个"(可选,时间够就做)

- 不关第一个客户端,**再开一个 `EchoClient` 跑一次** → 第二个连上后**没反应**。因为 `EchoServer` 的 `accept()` 只接了第一个,正忙着陪它,没回到 `accept()`。**这就是下一步图书馆服务端要用"每客户端一线程"解决的问题。**

卡住怎么办

- **客户端报 `Connection refused`(连接被拒)**:总机还没开——**先起服务端,再起客户端。**
- **服务端"没反应"**:那不是没反应,是 `accept()` 在等,去把客户端跑起来。
- **中文回复乱码**:两边读写流都要 `UTF-8`(`InputStreamReader(..., StandardCharsets.UTF_8)`)。

> 💡 **让 AI 当老师**:跑通后问它——"用中文逐行解释 EchoServer:ServerSocket 和 Socket 有什么区别,accept() 为什么会停住,readLine() 和 write()+flush() 分别在干什么。"对上母本 Part 1 的理论。

Step 1 · 建库 + 种子数据

目的:数据库是地基。先把四张表和测试数据准备好,后面服务端才有东西可查。

操作

- 在 Trae SOLO 里**原样复制**下面这段输入并回车:

```
按 .trae/rules 里的数据模型,给我一段 MySQL 建库脚本:库名 library_cs(DEFAULT CHARSET
utf8mb4),
建 t_user、t_category、t_book、t_borrow 四张表(字段照 rules,带主外键),插入种子数据:
3 个分类、6-8 本图书、管理员 admin/123456、读者 zhangsan/123456 和 lisi/123456。
只输出 SQL,我去 MySQL 执行。
```

- 把生成的 SQL 复制到 MySQL 客户端执行(或让 Trae 教你 `mysql -uroot -p12345678 <`

`init.sql`)。

- 执行后验证:`USE library_cs; SELECT * FROM t_book;`(看到 6-8 本书)、`SELECT * FROM t_user;`(看到 admin / zhangsan / lisi)。

预期产出(对照答案版 `sql/init.sql`,库名统一 `library_cs`)

```
CREATE DATABASE IF NOT EXISTS library_cs DEFAULT CHARSET utf8mb4 COLLATE
utf8mb4_general_ci;
USE library_cs;

DROP TABLE IF EXISTS t_borrow;
DROP TABLE IF EXISTS t_book;
DROP TABLE IF EXISTS t_category;
DROP TABLE IF EXISTS t_user;

-- 1. 用户表(读者 + 管理员)
CREATE TABLE t_user (
  id          INT PRIMARY KEY AUTO_INCREMENT,
  username    VARCHAR(50)  NOT NULL UNIQUE COMMENT '登录名',
  password    VARCHAR(100) NOT NULL        COMMENT '密码(教学项目明文, 生产应加密)',
  real_name   VARCHAR(50)  NOT NULL        COMMENT '真实姓名',
  role        VARCHAR(10)  NOT NULL DEFAULT 'reader' COMMENT 'reader / admin',
  max_borrow  INT          NOT NULL DEFAULT 5 COMMENT '可借上限',
  create_time DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='用户表';

-- 2. 图书分类表
CREATE TABLE t_category (
  id   INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(50) NOT NULL UNIQUE COMMENT '分类名'
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='图书分类表';

-- 3. 图书表
CREATE TABLE t_book (
  id          INT PRIMARY KEY AUTO_INCREMENT,
  isbn        VARCHAR(30)  NOT NULL DEFAULT '' COMMENT 'ISBN 号',
  title       VARCHAR(200) NOT NULL           COMMENT '书名',
  author      VARCHAR(100) NOT NULL DEFAULT '' COMMENT '作者',
  publisher   VARCHAR(100) NOT NULL DEFAULT '' COMMENT '出版社',
  category_id INT          NULL                COMMENT '分类外键',
  total       INT          NOT NULL DEFAULT 0  COMMENT '馆藏总数',
  stock       INT          NOT NULL DEFAULT 0  COMMENT '当前可借库存',
  CONSTRAINT fk_book_category FOREIGN KEY (category_id) REFERENCES t_category(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='图书表';

-- 4. 借阅记录表
CREATE TABLE t_borrow (
  id          INT PRIMARY KEY AUTO_INCREMENT,
  user_id     INT      NOT NULL COMMENT '借书人',
  book_id     INT      NOT NULL COMMENT '所借图书',
  borrow_time DATETIME NOT NULL COMMENT '借出时间',
```

```
  due_time    DATETIME NOT NULL COMMENT '应还时间',
  return_time DATETIME NULL     COMMENT '实际归还时间, NULL 表示未还',
  status      VARCHAR(10) NOT NULL DEFAULT 'borrowed' COMMENT 'borrowed /
returned',
  CONSTRAINT fk_borrow_user FOREIGN KEY (user_id) REFERENCES t_user(id),
  CONSTRAINT fk_borrow_book FOREIGN KEY (book_id) REFERENCES t_book(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='借阅记录表';

-- 种子:用户
INSERT INTO t_user (username, password, real_name, role, max_borrow) VALUES
  ('admin',    '123456', '系统管理员', 'admin',  99),
  ('zhangsan', '123456', '张三',       'reader', 5),
  ('lisi',     '123456', '李四',       'reader', 3);

-- 种子:分类
INSERT INTO t_category (name) VALUES ('计算机'), ('文学'), ('历史');

-- 种子:图书(category_id: 1=计算机 2=文学 3=历史)
INSERT INTO t_book (isbn, title, author, publisher, category_id, total, stock)
VALUES
  ('9787111213826', '深入理解计算机系统', 'Randal E. Bryant', '机械工业出版社', 1, 5,
5),
  ('9787115428028', 'Java核心技术卷I',    'Cay S. Horstmann', '人民邮电出版社', 1, 4,
4),
  ('9787121362217', '算法导论',           'Thomas H. Cormen', '电子工业出版社', 1,
3, 3),
  ('9787020024759', '红楼梦',             '曹雪芹',           '人民文学出版社', 2, 6,
6),
  ('9787532736553', '百年孤独',           '加西亚·马尔克斯', '上海译文出版社', 2, 2,
2),
  ('9787506365437', '活着',               '余华',             '作家出版社',     2,
3, 3),
  ('9787101003048', '史记',               '司马迁',           '中华书局',       3,
2, 2),
  ('9787108009821', '万历十五年',         '黄仁宇',           '生活·读书·新知三联书店',
3, 1, 1);
```

> 📌 `t_fine`(罚款表)是可选的,今天不需要,后面用到再加。今天四张核心表就够打通登录链路了。

✅ 怎么算做好

- SQL 在 MySQL 里执行无报错;`SELECT * FROM t_book;` 看到 6–8 本书,`SELECT * FROM t_user;` 看到 admin / zhangsan / lisi。

卡住怎么办

- **执行报外键错误**:建表顺序要先建被引用的表(`t_category` / `t_user`)再建 `t_book` / `t_borrow`;删表要反过来先删子表。让 AI 检查顺序。
- **中文插入变问号**:确认库和表都是 `utf8mb4`;MySQL 客户端连接也用 utf8mb4。
- **`library_cs` 已存在旧数据**:脚本里的 `DROP TABLE IF EXISTS` 会先清旧表再重建,重复执行也安全。

> ⚠️ **字符集两个位置别搞混:建库脚本里用** `utf8mb4`(MySQL 字符集名,数据库该用它);但下一步**服务端 JDBC 连接串**里要用 `characterEncoding=UTF-8`(Java 字符集名,不是 utf8mb4)。这是两回事,Step 2 会踩到。

Step 2 · 服务端骨架:Socket + 多线程 + DBUtil

目的:搭起服务端的"空壳"——能启动监听 9000、每来一个客户端开一个线程、能收到一行并回显(先不接业务)。先把通道立起来。

操作

- 在 Trae SOLO 里**原样复制**下面这段输入并回车:

```
帮我创建 Java 服务端(纯 Java 11,不用框架):
- DBUtil:JDBC 连 127.0.0.1:3306/library_cs(root/12345678),url 带
characterEncoding=UTF-8(不要写 utf8mb4)、
  useSSL=false、serverTimezone=Asia/Shanghai;提供 getConnection()。
- LibraryServer:ServerSocket 监听 9000,accept 到一个客户端就 new Thread(new
ClientHandler(socket)).start()——每客户端一个线程。
- ClientHandler:循环 readLine() 收一行请求,先原样回显(先不接业务)。
- 一个手写的 Json/编码工具类,把 List<Map> 编成一行文本(含 | 和换行要转义),不引第三方库。
先只搭骨架:能启动监听、能收到客户端一行并回显。分层放 server/、server/util/、
common/util/。
```

- 把 MySQL 驱动 jar 放进 `lib/` 并加入类路径,运行 `LibraryServer`。

预期产出(对照答案版真实代码)

① **`server/util/DBUtil.java`**(注意字符集参数,库名 `library_cs`):

```java
package server.util;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

/** 数据库工具类:负责获取 MySQL 连接。 */
public final class DBUtil {

    private DBUtil() {}

    // characterEncoding 要填 Java 字符集名 UTF-8,不能写 MySQL 的 utf8mb4,
    // 否则报 Unsupported character encoding;Connector/J 8 会据此在服务端使用
utf8mb4。
    private static final String URL =
            "jdbc:mysql://127.0.0.1:3306/library_cs"
          + "?useSSL=false&characterEncoding=UTF-8&serverTimezone=Asia/Shanghai";
    private static final String USER = "root";
    private static final String PASSWORD = "12345678";
```

```
    static {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
        } catch (ClassNotFoundException e) {
            throw new RuntimeException("未找到 MySQL 驱动,请确认 lib/ 下有 mysql-
connector-j jar", e);
        }
    }

    /** 获取一个新的数据库连接(调用方负责关闭)。 */
    public static Connection getConnection() throws SQLException {
        return DriverManager.getConnection(URL, USER, PASSWORD);
    }
}
```

② **`server/LibraryServer.java`**(监听 + 每客户端一线程,核心两行):

```
package server;

import java.io.IOException;
import java.net.ServerSocket;
import java.net.Socket;

/** 图书馆服务端:监听 9000,每 accept 到一个客户端就开一个线程去服务它。 */
public class LibraryServer {

    public static final int PORT = 9000;

    public static void main(String[] args) throws IOException {
        ServerSocket serverSocket = new ServerSocket(PORT);
        System.out.println("[Server] 启动成功,监听端口 " + PORT + ",等待客户端连
接...");
        while (true) {
            Socket client = serverSocket.accept();          // 阻塞等客户端
            new Thread(new ClientHandler(client)).start();   // 每客户端一线程
        }
    }
}
```

> 📌 答案版的 `LibraryServer` 还额外做了 `start()` / `stop()` 和监控窗口支持(单元④才需要),今天骨架阶段有上面这个 `main` 里的 `accept()` + `new Thread(...).start()` 循环就够。核心是那两行:`accept()` 阻塞等人、拿到连接立刻开线程。

③ **`server/ClientHandler.java`**(骨架阶段:收一行、回一行;流用 UTF-8):

```
package server;

import java.io.*;
import java.net.Socket;
import java.nio.charset.StandardCharsets;
```

```
/** 客户端处理线程 —— 每个客户端连接对应一个本类实例(一个线程)。 */
public class ClientHandler implements Runnable {

    private final Socket socket;

    public ClientHandler(Socket socket) {
        this.socket = socket;
    }

    @Override
    public void run() {
        try (BufferedReader in = new BufferedReader(
                    new InputStreamReader(socket.getInputStream(),
StandardCharsets.UTF_8));
             BufferedWriter out = new BufferedWriter(
                    new OutputStreamWriter(socket.getOutputStream(),
StandardCharsets.UTF_8))) {

            String line;
            while ((line = in.readLine()) != null) {    // 循环收一行请求
                String response = handle(line);          // 骨架阶段:先原样回显
                out.write(response);
                out.write("\n");
                out.flush();
            }
        } catch (IOException e) {
            // 客户端异常断开,静默即可
        } finally {
            try { socket.close(); } catch (IOException ignore) {}
        }
    }

    /** 骨架阶段先原样回显;Step 3 再改成按命令分发。 */
    private String handle(String line) {
        return "ECHO|" + line;
    }
}
```

④ **`common/util/Json.java`**(手写极简编码,把列表编成不含裸 `|`/换行的一行;下面是它的对外方法与转义思路,核心是 `esc`/`unesc`):

```
package common.util;

import java.util.*;

/**
 * 极简"表格"序列化工具(不引第三方 JSON 库,自己手写)。
 * 把 List<Map<String,String>> 编码成单行、且不含竖线 | 和换行 的字符串。
 * 之所以把 | 也转义掉,是因为外层协议按 | 拆分,编码结果里绝不能出现裸的 |。
 */
```

```
public final class Json {

    private Json() {}

    private static final char FS = '\u0002'; // 字段分隔
    private static final char RS = '\u0003'; // 记录分隔

    public static String encode(List<Map<String, String>> rows) {
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < rows.size(); i++) {
            if (i > 0) sb.append(RS);
            boolean first = true;
            for (Map.Entry<String, String> e : rows.get(i).entrySet()) {
                if (!first) sb.append(FS);
                first = false;
                sb.append(esc(e.getKey())).append('=').append(esc(e.getValue()));
            }
        }
        return sb.toString();
    }

    // 值里的 \ | = 换行 FS RS 全部转义,保证协议不被拆乱
    private static String esc(String v) {
        if (v == null) return "";
        StringBuilder sb = new StringBuilder(v.length() + 8);
        for (int i = 0; i < v.length(); i++) {
            char c = v.charAt(i);
            switch (c) {
                case '\\': sb.append("\\\\"); break;
                case '|':  sb.append("\\P");  break;
                case '=':  sb.append("\\E");  break;
                case '\n': sb.append("\\N");  break;
                case '\r': sb.append("\\C");  break;
                case FS:   sb.append("\\F");  break;
                case RS:   sb.append("\\R");  break;
                default:   sb.append(c);
            }
        }
        return sb.toString();
    }
    // 完整版还有 decode/decodeOne/unesc(反转义),供客户端解码;答案版
common/util/Json.java 是完整实现。
}
```

> 📌 今天骨架阶段还用不到 `Json`(登录返回是简单的 `OK|role|realName|userId`,不是列表)。它是为后面查书、借阅记录这类**列表数据**准备的。先生成放着,单元③查书时就派上用场。

✅ 怎么算做好

- 运行 `LibraryServer`,控制台打印"启动成功,监听端口 9000,等待客户端连接";用任意方式连上去发一行,收到回显(如 `ECHO|hello`)。

卡住怎么办(两个高频坑,一定盯)

- ⚠️ **坑 1:一连库就报 `Unsupported character encoding 'utf8mb4'`** 原因:`characterEncoding` 这个参数要填 **Java 的字符集名** `UTF-8`,`utf8mb4` 是 MySQL 的字符集名,Java 不认。用纠错三步法:

  ```
  DBUtil 的 JDBC url 里 characterEncoding 写成了 utf8mb4,一连库就报 Unsupported character encoding。
  请改成 characterEncoding=UTF-8(Java 字符集名),数据库层的 utf8mb4 不用动。只改这一处。
  ```

- ⚠️ **坑 2:第二个客户端连上来没反应,只能服务一个人**(就是 Step 0 你亲眼见过的现象) 原因:`accept()` 后没开线程,在主线程里直接处理了;或者开线程时**写成了** `run()` **而不是** `start()`(`run()` 不起新线程,只是普通调一次方法)。用纠错三步法:

  ```
  LibraryServer 现在 accept 到客户端后是在主线程里直接处理的,这样只能服务一个客户端。
  请改成:accept 到一个 socket 后 new Thread(new ClientHandler(socket)).start() 去处理,
  主线程立刻回到 accept 继续等下一个客户端。只改这一处,别的不动。
  ```

- **找不到 MySQL 驱动** `ClassNotFoundException`:确认 `lib/` 下有 `mysql-connector-j` jar 且加进了类路径。
- **中文回显变问号乱码**:客户端和服务端读写流都要用 `UTF-8`(`InputStreamReader(...., StandardCharsets.UTF_8)`)。

> 💡 **让 AI 当老师**:运行起来后问它——"用中文逐行解释 `accept()` 为什么会阻塞、为什么每来一个客户端要开一个新线程、不开会怎样。"看懂了你才审得动它。

Step 3 · 打通登录链路(今天最重要的一步)

目的:把"空壳"变成能干活的——实现 `LOGIN` 这一条完整链路:客户端发 `LOGIN|用户名|密码` → 服务端查 `t_user` → 返 `OK|role|realName|userId` 或 `ERR|账号或密码错误`。

操作

- 承接 Step 2,在 Trae SOLO 里**原样复制**下面这段输入并回车:

```
基于现有服务端,实现"登录"这一条完整链路:
- 定义协议:客户端发 LOGIN|用户名|密码;
- ClientHandler 解析命令,登录走 UserService.login → UserDao 用 PreparedStatement 查 t_user;
- 成功返回 OK|role|realName|userId,失败返回 ERR|账号或密码错误。
加 server/service/UserService、server/dao/UserDao、common/entity/User。只做登录。
```

预期产出(对照答案版真实代码)

① `common/entity/User.java`(用户实体,对应 t_user):

```
package common.entity;

/** 用户实体(读者 / 管理员)。对应 t_user 表。 */
public class User {
    private int id;
    private String username;
    private String password;
    private String realName;
    private String role;      // reader / admin
    private int maxBorrow;

    public User() {}

    public int getId() { return id; }
    public void setId(int id) { this.id = id; }
    public String getUsername() { return username; }
    public void setUsername(String username) { this.username = username; }
    public String getPassword() { return password; }
    public void setPassword(String password) { this.password = password; }
    public String getRealName() { return realName; }
    public void setRealName(String realName) { this.realName = realName; }
    public String getRole() { return role; }
    public void setRole(String role) { this.role = role; }
    public int getMaxBorrow() { return maxBorrow; }
    public void setMaxBorrow(int maxBorrow) { this.maxBorrow = maxBorrow; }

    public boolean isAdmin() { return "admin".equals(role); }
}
```

② **`server/dao/UserDao.java`**(⭐ 用 `PreparedStatement` 查库,防注入):

```
package server.dao;

import common.entity.User;
import server.util.DBUtil;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;

/** 用户 DAO:封装对 t_user 表的访问。所有 SQL 均使用 PreparedStatement 防注入。 */
public class UserDao {

    /** 按用户名 + 密码查询用户;查不到返回 null。 */
    public User findByUsernameAndPassword(String username, String password)
throws SQLException {
        String sql = "SELECT id, username, password, real_name, role, max_borrow
"
                   + "FROM t_user WHERE username = ? AND password = ?";
        try (Connection conn = DBUtil.getConnection();
```

```
             PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setString(1, username);
            ps.setString(2, password);
            try (ResultSet rs = ps.executeQuery()) {
                if (rs.next()) {
                    return map(rs);
                }
                return null;
            }
        }
    }

    private User map(ResultSet rs) throws SQLException {
        User u = new User();
        u.setId(rs.getInt("id"));
        u.setUsername(rs.getString("username"));
        u.setPassword(rs.getString("password"));
        u.setRealName(rs.getString("real_name"));
        u.setRole(rs.getString("role"));
        u.setMaxBorrow(rs.getInt("max_borrow"));
        return u;
    }
}
```

③ **`server/service/UserService.java`**(业务层:登录校验):

```
package server.service;

import common.entity.User;
import server.dao.UserDao;

import java.sql.SQLException;

/** 用户业务:登录。 */
public class UserService {

    private final UserDao userDao = new UserDao();

    /** 登录校验;成功返回 User,失败返回 null。 */
    public User login(String username, String password) throws SQLException {
        return userDao.findByUsernameAndPassword(username, password);
    }
}
```

④ **`ClientHandler` 里加 `LOGIN` 分支**(把骨架的"回显"改成"按命令分发"):

```
// ClientHandler 里:handle() 改成按命令分发,并加 doLogin
private final UserService userService = new UserService();

private String handle(String line) {
    try {
```

```
            String[] p = line.split("\\|", -1);   // 按 | 拆成 命令 + 参数
            String cmd = p[0];
            switch (cmd) {
                case "LOGIN": return doLogin(p);
                // 后面单元再往这里加 BOOK_QUERY / BORROW / RETURN ...
                default:      return "ERR|未知命令: " + cmd;
            }
        } catch (Exception e) {
            return "ERR|服务端异常: " + e.getMessage();
        }
    }

    private String doLogin(String[] p) throws Exception {
        String username = p.length > 1 ? p[1] : "";
        String password = p.length > 2 ? p[2] : "";
        User u = userService.login(username, password);
        if (u == null) return "ERR|账号或密码错误";
        // OK|role|realName|userId
        return "OK|" + u.getRole() + "|" + u.getRealName() + "|" + u.getId();
    }
```

> 📌 答案版 `ClientHandler` 用了 `Protocol` 常量类(`Protocol.LOGIN`、`Protocol.OK` 等)和一整套命令分支,思路完全一样——今天只需要 `LOGIN` 这一条通了即可,其余命令后面单元逐步加。

✅ 怎么算做好(Step 3 的验收点)

- 起服务端,发 `LOGIN|admin|123456` 收到 `OK|admin|系统管理员|1`;发 `LOGIN|admin|000000` 收到 `ERR|账号或密码错误`。**这条 C/S 通道打通了,后面全是复制它换业务。**

怎么发这一行、怎么看结果

- **办法一(推荐):让 AI 写个最简单的测试客户端。** SOLO 输入:

  ```
  给我一个 5 行左右的 Java 命令行小程序(或一段可直接跑的 main),连 127.0.0.1:9000,
  发一行 LOGIN|admin|123456,把服务端返回的一行打印出来。只要能验证登录链路,不用界面。
  ```

  运行它,控制台应打印 `OK|admin|系统管理员|1`;把账号密码改错再跑,应打印 `ERR|账号或密码错误`。
- **办法二:先跳到单元③用真客户端**(想直接看界面再登录时用)。今天用办法一就足够证明链路通了。

卡住怎么办

- **返回 `ERR|账号或密码错误` 但账号没错**:确认 Step 1 的种子数据插进去了(`SELECT * FROM t_user WHERE username='admin';`);确认 `UserDao` 的 SQL 字段名和表一致(`real_name` / `max_borrow`)。
- **报 SQL 异常连不上库**:回到 Step 2 坑 1(`characterEncoding=UTF-8`)、确认库名是 `library_cs`、MySQL 已启动。
- **中文姓名返回乱码**:客户端服务端流都要 UTF-8;`DBUtil` 的连接串 `characterEncoding=UTF-8`。

Step 4 · 顺手闭合:REGISTER 注册链路 · [信任但验证] [规范优先]

目的:登录通道刚打通,但库里的读者账号(zhangsan/lisi)都是 Step 1 种子数据**手插**的。真实场景读者得能**自己注册**。好消息:注册和登录是**同一条通道换业务**——客户端发 `REGISTER|用户名|密码|姓名` → 服务端**查重 + 插库** → 返 `OK|新用户id`。你已经会登录了,注册就是顺手闭合,再练一遍"复制链路换命令换 SQL"。这一步比登录多一件事:**写库前要先查重**(用户名不能重复)——这是你审查 AI 的新看点。

操作

- 承接 Step 3 的**同一对话**,在 Trae SOLO 里**原样复制**下面这段输入并回车:

```
基于现有服务端的登录链路,再补一条"读者注册"完整链路,协议命令用 REGISTER:
- 协议:客户端发 REGISTER|用户名|密码|真实姓名,成功返回 OK|新用户id,失败返回 ERR|原因;
  在 common/Protocol 里加 REGISTER 常量。
- 服务端:ClientHandler 加 REGISTER 分支 → UserService.register:
  先校验用户名和密码非空,再用 UserDao.existsUsername 查重(用户名已存在就抛异常提示"用户名已存在"),
  通过后 UserDao.insertReader 插入,role 固定 'reader'、max_borrow 默认 5(注册的一律是读者,不能是 admin),
  返回自增主键。所有 SQL 都用 PreparedStatement。
- 客户端:一个 Swing 注册对话框 RegisterDialog(用户名/密码/确认密码/真实姓名四个输入框),
  提交前在界面上校验"用户名密码非空""两次密码一致",再调 Api.register 发 REGISTER;
  成功弹"注册成功,请用新账号登录"。
t_user 的 username 已是 UNIQUE 唯一索引(Step 1 建库脚本里有),注册重名要走 existsUsername 提前拦。
```

预期产出(对照答案版 `cs-swing-library-demo` 真实代码)

① **`common/Protocol.java`** 加一行命令常量(和 LOGIN 并排):

```
public static final String LOGIN    = "LOGIN";    // LOGIN|username|password
        -> OK|role|realName|userId
public static final String REGISTER = "REGISTER"; //
REGISTER|username|password|realName -> OK|userId
```

② **`server/dao/UserDao.java`** 新增**查重** + **插入**两个方法(都用 `PreparedStatement`):

```
/** 判断用户名是否已存在。 */
public boolean existsUsername(String username) throws SQLException {
    String sql = "SELECT 1 FROM t_user WHERE username = ?";
    try (Connection conn = DBUtil.getConnection();
         PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setString(1, username);
        try (ResultSet rs = ps.executeQuery()) {
            return rs.next();
        }
    }
```

```
}

/** 新增读者,返回自增主键。 */
public int insertReader(String username, String password, String realName, int
maxBorrow) throws SQLException {
    String sql = "INSERT INTO t_user(username, password, real_name, role,
max_borrow) "
                + "VALUES(?, ?, ?, 'reader', ?)";
    try (Connection conn = DBUtil.getConnection();
         PreparedStatement ps = conn.prepareStatement(sql,
Statement.RETURN_GENERATED_KEYS)) {
        ps.setString(1, username);
        ps.setString(2, password);
        ps.setString(3, realName);
        ps.setInt(4, maxBorrow);
        ps.executeUpdate();
        try (ResultSet keys = ps.getGeneratedKeys()) {
            if (keys.next()) return keys.getInt(1);
        }
        return 0;
    }
}
```

> 圈重点:① 插入 SQL 把 `role` **写死成** `'reader'`——注册进来的一律是读者,**绝不能让人注册成 admin**;`max_borrow` 传 5(新读者默认可借 5 本)。② `Statement.RETURN_GENERATED_KEYS` + `getGeneratedKeys()` 拿回**自增主键**返给客户端。

③ **`server/service/UserService.java`** 的 `register`(业务校验都在这一层):

```
/**
 * 读者注册。
 * @return 新用户 id
 * @throws IllegalArgumentException 用户名已存在或参数非法
 */
public int register(String username, String password, String realName) throws
SQLException {
    if (username == null || username.trim().isEmpty()
            || password == null || password.isEmpty()) {
        throw new IllegalArgumentException("用户名和密码不能为空");
    }
    if (userDao.existsUsername(username.trim())) {
        throw new IllegalArgumentException("用户名已存在");
    }
    String name = (realName == null || realName.trim().isEmpty()) ?
username.trim() : realName.trim();
    // 新注册读者默认可借上限 5 本
    return userDao.insertReader(username.trim(), password, name, 5);
}
```

> 圈重点:**先非空校验 → 再查重 → 最后插入**,顺序不能反。查重命中抛 `IllegalArgumentException("用户名已存在")`,`ClientHandler` 外层会把它转成 `ERR|用户名已存在` 回给客户端;姓名留空就用用户名兜底。

④ **`server/ClientHandler.java`** 里 `handle()` 加 `REGISTER` 分支 + `doRegister`(和 `doLogin` 并排):

```
switch (cmd) {
    case Protocol.LOGIN:    return doLogin(p);
    case Protocol.REGISTER: return doRegister(p);   // ← 新增这一行
    default:                return err("未知命令: " + cmd);
}

private String doRegister(String[] p) throws Exception {
    int id = userService.register(arg(p, 1), arg(p, 2), arg(p, 3));
    return Protocol.OK + "|" + id;      // OK|新用户id
}
```

> 圈重点:`handle()` 外层 `catch (IllegalArgumentException e) { return err(e.getMessage()); }` 把业务异常("用户名已存在"/"用户名和密码不能为空")统一转成 `ERR|原因`——所以 Service 里才敢直接 `throw`。

⑤**客户端两段**(单元③接界面时细讲,今天先看真码对上号):`client/net/Api.java`:

```
public static int register(String username, String password, String realName)
throws ApiException {
    String payload = send(Protocol.buildRequest(Protocol.REGISTER, username,
password, realName));
    return toInt(payload);
}
```

`client/ui/RegisterDialog.java` 提交时的**界面前置校验**:

```
private void doRegister() {
    String username = usernameField.getText().trim();
    String password = new String(passwordField.getPassword());
    String confirm = new String(confirmField.getPassword());
    String realName = realNameField.getText().trim();

    if (username.isEmpty() || password.isEmpty()) {
        JOptionPane.showMessageDialog(this, "用户名和密码不能为空", "提示",
JOptionPane.WARNING_MESSAGE);
        return;
    }
    if (!password.equals(confirm)) {
        JOptionPane.showMessageDialog(this, "两次输入的密码不一致", "提示",
JOptionPane.WARNING_MESSAGE);
        return;
    }
    Async.run(this, () -> Api.register(username, password, realName), newId -> {
```

```
            JOptionPane.showMessageDialog(this, "注册成功!请用新账号登录。", "成功",
JOptionPane.INFORMATION_MESSAGE);
            dispose();
        });
}
```

> 🔑 **一条关键认知**:界面校验(非空 / 两次密码一致)是**体验**,服务端校验(非空 / 查重)是**安全底线——两层都要**。界面校验只是少一次白跑网络;懂行的人能绕过界面直接发协议,**服务端那道校验才真正拦得住**。这就是"信任但验证"在一次注册里的样子。

✅ 怎么算做好(端到端闭合:注册 → 用新账号登录 → 收 OK)

用 Step 3 的测试小程序(把要发的那行改一下)逐条验:

- [ ] 发 `REGISTER|wangwu|abc123|王五`,收到 `OK|4`(新用户自增 id,数字随库而定)。
- [ ] `SELECT * FROM t_user WHERE username='wangwu';`——看到新行,`role='reader'`、`max_borrow=5`。
- [ ] **端到端闭合**:紧接着发 `LOGIN|wangwu|abc123`,收到 `OK|reader|王五|4`——**刚注册的账号能登录了,一条链路从头闭合。**
- [ ] 再发一次 `REGISTER|wangwu|xxx|王五2`(重名),收到 `ERR|用户名已存在`(不是数据库报错栈)。
- [ ] 发 `REGISTER|zhaoliu||赵六`(密码空),收到 `ERR|用户名和密码不能为空`。

卡住怎么办(踩坑表:问题 | 原因 | 追问 prompt)

现象	原因	追问 prompt(纠错三步法:定位→描述→只改一处)
重名注册返回一长串 `ERR|服务端异常: Duplicate entry 'wangwu' for key...`	Service 没先查重就插库,撞上 `username` UNIQUE 唯一索引,MySQL 抛异常被当成"服务端异常"	`注册重名时返回的是 Duplicate entry 数据库异常栈,用户看不懂。请在 UserService.register 里 insertReader 之前先调 userDao.existsUsername 查重,命中就 throw new IllegalArgumentException("用户名已存在"),让 ClientHandler 转成 ERR|用户名已存在。只改 register 这一处。`
忘了给 `username` 加 UNIQUE,重名能插进两条	Step 1 建库脚本 `username` 列漏了 `UNIQUE`;并发下两个请求可能同时通过查重双双插入(查重不是原子的)	`t_user 的 username 列没有 UNIQUE 唯一索引,重名能插进两条。请确认建库脚本里 username 是 VARCHAR(50) NOT NULL UNIQUE;唯一索引是并发下最后一道防线,Service 查重挡不住并发。只改建表 username 那一列。`
注册成功但 `role` 是空 / 能注册出 admin	insertReader 的 SQL 没写死 `role='reader'`,或把 role 当参数从客户端传进来	`注册插入时 role 不能由客户端决定,否则有人能注册成 admin。请把 insertReader 的 INSERT 里 role 固定写成 'reader',不接收外部 role 参数。只改这条 INSERT。`
界面两次密码不一致也发了请求	RegisterDialog 提交前没校验 `password.equals(confirm)`	`RegisterDialog 提交时没校验两次密码一致就发请求了。请在 doRegister 里、调 Api.register 之前加:两次密码不一致就 JOptionPane 提示"两次输入的密码不一致"并 return。只改 doRegister。`

迭代第二步(同一对话继续加需求:学号必填 + 邮箱格式)

注册跑通后,真实场景往往要读者填**学号**并校验**邮箱格式**。承接同一对话继续追加(这次没有答案版真码,按"预期行为"验收):

```
在现有注册链路上加两个字段校验:
1. t_user 加一列 student_id VARCHAR(20)(学号),给它也加 UNIQUE 唯一索引;
   RegisterDialog 加"学号"输入框,注册时学号必填、且不能和已有学号重复(服务端也查重)。
2. RegisterDialog 加"邮箱"输入框,提交前用正则校验邮箱格式(不合法就提示"邮箱格式不正确"并拦下);
   邮箱存进 t_user 新列 email。
协议扩成 REGISTER|用户名|密码|姓名|学号|邮箱,服务端 UserService.register 同步接这两个参数,
学号查重复用 existsUsername 的思路新写一个 existsStudentId。所有校验服务端也要有一份(界面校验只是体验)。
```

- **预期行为**:学号留空 → 界面拦 `学号必填`;学号重复 → 服务端返 `ERR|学号已存在`;邮箱写 `abc@`(不合法)→ 界面拦 `邮箱格式不正确`,请求根本不发出。
- ⚠️ **审查看点**:AI 大概率把邮箱正则**只放界面**。追问它:"服务端 register 是否也校验了邮箱和学号?界面能被绕过。"逼它把校验补到 Service 层(呼应前面"两层都要")。

> 💡 **让 AI 当老师**:跑通后问它——"用中文解释:为什么注册要在 Service 层先 existsUsername 查重,数据库已经有 UNIQUE 唯一索引了还查重是不是多余?"(答案:查重给用户友好提示,唯一索引兜并发,两者分工不同,都要。)

> ✓ **这一步练了什么**:① **信任但验证**——审出"AI 只在界面校验、服务端裸奔"这个高频漏,知道"唯一索引是并发最后一道防线";② **规范优先**——REGISTER 协议格式、role 写死 reader、username UNIQUE 都是单元① PRD 的规范,照规范生成、照规范验收;③ **复制链路换业务**——注册和登录同一条 socket 通道、同一套线程,只换了命令 / SQL / 返回。**你没学新的 Socket 知识,真正新练的是"写库前查重""校验两层""唯一索引兜底"这几条把关功夫。**

今天的审查清单(你的核心价值)

AI 生成得快,但你不是看客。对照下面四条逐一审(在单元①图纸里你已写死它们,现在拿来验收):

- [ ] **架构没退回档 A**:数据库地址密码只在服务端 `DBUtil` 里,客户端不碰库。
- [ ] **`characterEncoding=UTF-8` 而不是 utf8mb4**:连接串这个参数写对了。
- [ ] **`accept()` 后开了线程**:`new Thread(new ClientHandler(socket)).start()` 在,是 `start()` 不是 `run()`,不是主线程直接处理。
- [ ] **查库用 `PreparedStatement`(`?` 占位)**:`UserDao` 没有拼字符串。

发现不对,用**纠错三步法**(定位 → 描述 → 只改一处)让 AI 补。**让 AI 干活、你把关,这就是"信任但验证"。**

今天做完了什么(自检清单)

- [ ] 看懂 Socket / 线程两块基础,跑通 echo demo(服务端收到什么原样发回)(Step 0)
- [ ] 建好 `library_cs` 库 + 四张表 + 种子,能 `SELECT` 出图书和三个用户(Step 1)
- [ ] 服务端能启动监听 9000、每客户端一线程、收一行回显(Step 2)
- [ ] 打通登录链路:发 `LOGIN|admin|123456` 收到 `OK|admin|系统管理员|1`(Step 3)

- [ ] 顺手闭合注册链路:注册新读者 `REGISTER|wangwu|abc123|王五` 收 `OK|id`,再用它登录收到 `OK`;重名/空密码收到 `ERR|...`(Step 4)
- [ ] 对照四条审查清单验收过 AI 的代码

四项都打勾,今天目标就达成了——**C/S 那条命脉通道(客户端 → 服务端 → 数据库 → 返回)真正打通了。**

课后任务

1. 把今天的登录链路**再走一遍并画出来**:用一张图标出 `LOGIN|admin|123456` 从客户端出发,经 `ClientHandler` → `UserService` → `UserDao` → `t_user` → 返回 `OK|admin|系统管理员|1` 的完整路径,每一步标"谁在干什么"。
2. **主动制造一个坑再修好**:把 `DBUtil` 的 `characterEncoding` 故意改成 `utf8mb4`,跑一次看报什么错,再用纠错三步法改回来。亲手踩一次,你就永远记住这个坑了。
3. 让 AI 用大白话给你讲清楚:**为什么 `UserDao` 用 `?` 占位就能防 SQL 注入,拼字符串会怎样被攻击。**把它的解释记下来,下次课带着这个理解去审借书那段 SQL。

> 记住今天最重要的一句话:**先通一条链路,再复制换业务。** 登录这条通道打通了,你就掌握了整个 C/S 的骨架。